

并行计算 II 2026 春第二次作业

基于 MPI 的矩阵-向量乘法并行实现

王越洋

1 矩阵-向量乘法

矩阵-向量乘法 (Matrix-Vector Multiplication, 常缩写为 GEMV) 是稠密线性代数中最基本的运算之一, 广泛出现在迭代求解器 (如 Krylov 子空间法、共轭梯度法)、图分析 (PageRank)、深度学习推理等场景中。

给定一个 $N \times N$ 的矩阵 A 和一个长度为 N 的向量 x , 矩阵-向量乘法计算

$$y = A \cdot x, \quad y_i = \sum_{j=0}^{N-1} A_{ij} x_j, \quad i = 0, 1, \dots, N-1.$$

1.1 算法流程

串行算法的时间复杂度为 $O(N^2)$, 串行代码如下:

```
for (long i = 0; i < N; ++i) {  
    long long s = 0;  
    for (long j = 0; j < N; ++j)  
        s += A[i][j] * x[j];  
    y[i] = s;  
}
```

确定性数据初始化 为了便于自动化批改与跨机器复现, 本作业要求矩阵 A 与向量 x 均由 **确定性公式**生成, 不允许使用 `rand()` 等随机数:

$$A_{ij} = (i \cdot 7 + j \cdot 13) \bmod 1000, \quad x_j = (j \cdot 3 + 1) \bmod 100.$$

由此得到的 y 是确定的 64 位整数序列, 助教提供的验证脚本 `verify.py` 可对 y 做**逐元素精确比对**, 不存在浮点公差问题。

1.2 并行化策略

注意到不同的 y_i 之间是**完全独立**的, 因此矩阵的行天然适合作为并行任务的最小单位。下面列举三种典型的数据划分方式。

1D 行块划分 (推荐) 将 A 的 N 行按块划分给 p 个进程，进程 r 持有第 $start_r$ 到 $end_r - 1$ 行 (共 $local_rows_r$ 行)，每个进程额外保留一份完整的 x 。每个进程独立计算 $local_y_r$ ，最后用 `MPI_Gather` 或 `MPI_Gatherv` 汇集到 0 号进程。通信轮数仅为 1，实现最简单，是本作业推荐的方案。

1D 列块划分 将 A 按列划分， x 同样按列划分。每个进程计算部分和后用 `MPI_Reduce` (或 `MPI_Allreduce`) 求和。本方案通信量较大 (要 reduce 一个长度为 N 的向量)，仅作参考。

2D 块划分 (拓展) 将 $p = \sqrt{p} \times \sqrt{p}$ 个进程排成方阵， A 切成二维块 (Fox / SUMMA 算法)。通信量最优但实现较复杂，作为有兴趣同学的拓展方向。

数据划分不整除的处理 若 N 不被进程数 p 整除，则必须使用 `MPI_Gatherv`，并显式构造 `recvcounts[]` 与 `displs[]` 数组，否则会出现“接收长度对不齐”的运行时错误或结果错位。本作业的算例 M (见下表) **特意设计为不被 16 整除**，就是为了考察这一情形。

2 作业要求

根据附件中给定的串行实现 `serial.cpp` 与并行模板 `parallel.cpp`，补全 `parallel.cpp` 中所有 `/* Fill in your code */` 处的代码，也可重新修改结构，并完成下表中的三个算例：

算例	N	主要考察目的	是否整除 16	推荐通信原语	节点总内存
S	32	正确性与可读性 / 截图	是	<code>MPI_Gather</code>	8 KB
M	1000	不整除考点 / 自动验证	否	<code>MPI_Gatherv</code>	8 MB
L	8192	性能与加速比测量	是	<code>MPI_Gather</code>	512 MB

具体要求如下：

- **算例 S ($N = 32$)** —— **正确性**：运行后将 y 的输出与串行版本逐项对照，截图作为正确性证据，附入报告。
- **算例 M ($N = 1000$)** —— **Gatherv 考点**： N 不被 16 整除，**必须使用 `MPI_Gatherv`**。将 `slurm` 输出文件喂给助教提供的 `verify.py`，要求得到 `PASS`，否则视为该算例不通过。
- **算例 L ($N = 8192$, 默认)** —— **性能**：分别运行串行版与 16 进程并行版，每个版本 **至少独立运行 5 次取中位数**，在报告中给出运行时间表与加速比 $S = T_{\text{serial}}/T_{\text{parallel}}$ ，并讨论加速比偏离理想 $16\times$ 的原因。**若所用集群节点的可用内存充裕**，可以改用更大的 N (如 $N = 16384$ 或 $N = 32768$) 使加速比信号更稳定；反之若集群较紧张也可下调至 $N = 4096$ 。在报告中说明所选 N 即可，不强制使用 $N = 32768$ 。
- **矩阵的初始化必须由公式 $A_{ij} = (i \cdot 7 + j \cdot 13) \bmod 1000$ 在每个进程的本地完成**，不允许在 0 号进程上先构造完整 A 再 `MPI_Scatter` (这违背“并行初始化”的精神，且 0 号进程会率先 OOM)。

- 基于教学集群，最多使用 1 个节点与 16 个 MPI 进程。
- 推荐使用 C/C++ 实现，不允许使用 MPI 之外的并行实现库（如 OpenMP / Pthreads / CUDA 等）。
- 计时必须使用 MPI_Wtime 配合 MPI_Barrier，标准写法见下；不要使用 clock() / std::chrono，否则各进程间的初始化抖动会让数据不可比：

```
MPI_Barrier(MPI_COMM_WORLD);
double t0 = MPI_Wtime();
/* ---- 真正要测的计算与通信部分 ---- */
MPI_Barrier(MPI_COMM_WORLD);
double t1 = MPI_Wtime();
if (rank == 0)
    std::cout << "time=" << (t1 - t0) << "s\n";
```

- 输出格式：在 0 号进程上，将最终的 y 写入文件 $y_{N \times N}.txt$ ，每行一个整数（共 N 行）。模板代码中此段已写好，无需修改。然后将该文件作为参数喂给 verify.py 自动验证：

```
python3 verify.py y_N1000.txt
# PASS: all 1000 elements of y match the reference.
```

- 提交内容包括：补全后的 parallel.cpp、运行时间表与加速比图表（算例 L），以及一份报告说明划分方法、通信选择与性能分析。

拓展题（加分项） 选定一个 $N \geq 8192$ （推荐与算例 L 相同；集群允许时可使用更大的 N ，如 16384 或 32768，加速比信号会更稳定），在进程数 $p \in \{1, 2, 4, 8, 16\}$ 下分别运行并行版，绘制**强缩放（Strong Scaling）加速比曲线** $S(p) = T(1)/T(p)$ 与并行效率曲线 $E(p) = S(p)/p$ ，并结合“矩阵-向量乘法是访存密集型”的事实，分析曲线偏离理想加速的原因。

3 提交要求和评分标准

- **提交：**将代码和报告打包后发送至邮箱：wangyueyang@stu.pku.edu.cn，邮件主题为“学号-姓名-第二次作业”，截止日期为 6 月 1 日 23:59。
- **评分：**
 - (1) **基本要求：**补全 parallel.cpp，三个算例全部跑通；算例 M 在 verify.py 下得到 PASS；报告中给出运行截图、运行时间表、加速比与简要分析。
 - (2) **性能分析：**算例 L 的加速比越接近理论上限分数越高；报告中需说明数据划分方式与通信原语选择，并讨论加速比为何达不到 $16\times$ （参考“矩阵-向量乘法是访存密集型”这一事实）。
 - (3) **其他加分项：**完成拓展题（强缩放扫描），根据报告内容丰富程度、代码质量酌情加分。